Intel Quick Sync (GPU) Encoding for HandBrake Queue

Root cause analysis, fix and test results • asus-serv-1 (i3-10110U / UHD 620) • 10 November 2026

1. Summary

HandBrake Queue on asus-serv-1 encoded only on the CPU (software x264, about 30 fps), although the host has an Intel UHD 620 iGPU with Quick Sync and `/dev/dri` is passed through to the container. HandBrake reported *"qsv: not available on this system"* and listed no `qsv_*` encoders.

There were **two independent root causes**: (1) Ubuntu 26.04 only ships the Intel VPL GPU runtime, which supports 11th-gen (Tiger Lake) and newer, while the i3-10110U is 10th gen (Comet Lake); (2) Ubuntu's `handbrake-cli` package is linked against the system FFmpeg, which lacks a HandBrake/Intel QSV patch, so every QSV encode aborts in the muxer.

The fix builds HandBrakeCLI 1.11.2 from the upstream source (bundled, patched FFmpeg) and adds Intel's legacy Media SDK runtime. On a 2-minute test segment, Quick Sync H.264 encoded **about 2.2x faster** than x264 at the same bitrate and file size, with slightly lower SSIM.

Metric	Before (x264)	After (qsv_h264)
Encode speed (2-min 1080p HEVC 10-bit segment)	~29 fps (100 s)	~62 fps (46 s)
Estimated time, 43-min episode	~34 min	~16 min
Output size at 1500 kbps	27.3 MB	27.2 MB
SSIM / PSNR vs source	0.9876 / 45.85 dB	0.9854 / 45.08 dB

2. Environment

Item	Value
Host	asus-serv-1 (10.1.1.115), ASUS Chromebox 4, Ubuntu 26.04.1 LTS, Docker CE
CPU / GPU	Intel Core i3-10110U (2C/4T, Comet Lake) / Intel UHD Graphics 620 (Gen9.5)
App	HandBrake Queue (Handbrake-UI), container `handbrake-ui`, ~/SW/handbrake-ui, port 8095
GPU access	`/dev/dri` passed through; container user in group 991 (render); VA-API driver iHD loads OK
HandBrake (before)	Ubuntu package handbrake-cli 1.11.0, linked to system libavcodec.so.62
Workload	1080p HEVC 10-bit TV episodes to 720p H.264, preset "Gerry Medium TV Series 720" (x264 fast, 1500 kbps ABR, decomb, 30 fps peak)
Media	Sources: NFS from rasp-serv-1 NVMe; outputs: Synology DS418 NFS share

3. Root Cause 1: No GPU runtime for 10th-gen Intel

HandBrake finds hardware encoders through the oneVPL dispatcher (libvpl). The only GPU runtime in Ubuntu 26.04 is `libmfx-gen1.2` (Intel VPL GPU Runtime), which Intel built for Tiger Lake and newer. Gen9/Gen9.5 iGPUs (Skylake to Comet Lake) need the legacy Intel Media SDK runtime `libmfxhw64.so.1`, which is no longer packaged in 26.04. The dispatcher finds no usable implementation, so HandBrake disables QSV.

```
[18:24:19] qsv: not available on this system
libva info: va_openDriver() returns 0          <- VA-API itself works
```

Fix: install `libmfx1` 22.5.4 (Intel's final Media SDK release) from Ubuntu 24.04. Its only notable dependency is libva2 >= 2.9, which 26.04 satisfies. With it installed, HandBrake reports:

```
[18:29:37] qsv: is available on this system
Impl mfxhw64 library path: /usr/lib/x86_64-linux-gnu/libmfxhw64.so.1.35
   qsv_h264   qsv_h265   qsv_h265_10bit
```

4. Root Cause 2: Ubuntu's HandBrake lacks a QSV FFmpeg patch

With QSV detected, every `qsv_h264` encode still failed after about 1,100 frames. HandBrake builds the decode timestamps (DTS) itself, using the B-frame reorder depth that the encoder reports at start-up (`context->has_b_frames`, libhb/encavcodec.c). HandBrake enables B-pyramid for QSV by default, which needs two levels of reordering, but the unpatched FFmpeg encoder does not report the real depth. The timestamps come out out of order and the Matroska muxer aborts the job.

```
[matroska] Application provided invalid, non monotonically increasing dts to muxer
          in stream 0: 45562 >= 45478          (x ~800)
sync: got 1138 frames, 2901 expected
libhb: work result = 4
Encode failed (error 4).
```

HandBrake's official builds bundle their own FFmpeg with Intel's patch *"libavcodec/qsvenc.c: update has_b_frames value after initialization of encoder"* (contrib/ffmpeg/A16 in tag 1.11.2), which sets `has_b_frames = GopRefDist - 1`. Ubuntu's package is linked to the unpatched system `libavcodec.so.62`. The encoder options `b-pyramid=0` and `gop-ref-dist=1` had no effect (byte-identical output), so this cannot be worked around through settings.

Fix: build HandBrakeCLI from the upstream source, which compiles and links its patched FFmpeg.

5. Alternatives Considered

Option	Result	Decision
Move the app to rasp-serv-1 (Raspberry Pi 5, 8 GB)	21.6 fps with x264 (vs ~30 fps); ~48 min per episode. No H.264 hardware encoder on Pi 5. Other tools were unaffected (nice 10), 62.6 C peak, no throttling.	Rejected: slower, and no path to hardware encoding
Ubuntu handbrake-cli + legacy runtime only	QSV detected, but every encode fails (Root Cause 2)	Rejected
Encoder options (b-pyramid=0, gop-ref-dist=1)	No effect on output	Rejected
QSV H.264 "quality" preset	Identical to "balanced" on this GPU (46 s, SSIM 0.9854)	Not needed
QSV H.265 at 1500 kbps	55 s, SSIM 0.9844 (worse than both H.264 variants)	Not used
Upstream HandBrakeCLI 1.11.2 + legacy runtime	62 fps, 0 DTS errors, all frames encoded	**Adopted**

6. Test Method and Results

All tests used the same 2-minute segment (10:00 to 12:00) of a 1080p HEVC 10-bit episode, encoded to 720p at 1500 kbps average with the production preset settings, in throwaway containers. Quality was measured with FFmpeg SSIM and PSNR against the source scaled to 1280x720 (bicubic). The production queue was idle during the asus-serv-1 runs, so the timings are not affected by other load.

Variant	Wall time	Speed	Size	SSIM (All)	PSNR avg	Result
x264 fast (current)	100 s	~29 fps	27.3 MB	0.9876	45.85 dB	OK
qsv_h264 balanced	46 s	~62 fps	27.2 MB	0.9854	45.08 dB	OK
qsv_h264 quality	46 s	~62 fps	27.1 MB	0.9854	-	OK

qsv_h265 balanced	55 s	~52 fps	27.1 MB	0.9844	-	OK
qsv_h264 + QSV decoding	-	-	-	-	-	Fails at start
Pi 5, x264 fast (reference)	138 s	21.6 fps	-	-	-	OK

■ **NOTE:** The SSIM difference (0.002) should not be visible at normal viewing. To match x264 exactly, raise the QSV preset bitrate by about 10-15% (e.g. 1700 kbps), at the cost of slightly larger files.

The speed-up is about 2.2x rather than more, because the steps around the encoder still run on the CPU: decoding the HEVC 10-bit source, decomb/deinterlace and scaling.

7. Implementation

The project Dockerfile replaces the Ubuntu `handbrake-cli` package with two new build stages and an updated runtime stage:

Stage	What it does
handbrake	Clones HandBrake tag `HANDBRAKE_VERSION` (1.11.2) and runs `./configure --disable-gtk --enable-qsv --disable-nvenc --launch` (QSV only on amd64). Independent of the app source, so Docker caches it.
msdk	On amd64, downloads `libmfx1` (Media SDK runtime) from Ubuntu 24.04.
runtime	Installs the shared libraries HandBrakeCLI needs, the Intel VA-API driver and VPL runtime, and libmfx1; copies the binary. Fails the build if `ldd` reports a missing library.

```
RUN QSV=$([ "$TARGETARCH" = "amd64" ] && echo --enable-qsv || echo --disable-qsv) \
 && ./configure --disable-gtk $QSV --disable-nvenc --launch-jobs="$(nproc)" --launch \
 && install -m 755 -s build/HandBrakeCLI /usr/local/bin/HandBrakeCLI
```

File	Change
Dockerfile	New handbrake and msdk stages; runtime installs libraries instead of handbrake-cli
preset-gerry-qsv.json (new)	"Gerry Medium TV Series 720 QSV": same settings, qsv_h264 balanced, 1500 kbps
scripts/deploy.sh	Optional SSH_OPTS, e.g. `SSH_OPTS="-J rasp-web"` to deploy through the jump host
README.md	Hardware encoding section rewritten; new preset and SSH_OPTS documented

8. Deployment and Rollback

Step 1 - Build the image detached on asus-serv-1

The first build compiles HandBrake and its bundled libraries (about 45-60 min on 4 threads). Running it detached means a dropped SSH session cannot stop it, and the old container keeps serving the app meanwhile.

```
cd ~/SW/handbrake-ui && nohup docker compose build > hb-build.log 2>&1 &
```

Step 2 - Swap the container (only when no job is encoding)

deploy.sh refuses to run while a job is converting. The build is cached, so this takes seconds.

```
SSH_OPTS="-J rasp-web" scripts/deploy.sh
```

Step 3 - Verify and switch the preset

Check that `/api/info` lists qsv_h264, import preset-gerry-qsv.json (Presets page or POST /api/presets) and make it the default. The x264 preset stays as a fallback.

Status when this report was generated: code changes complete (uncommitted); deploy in progress.

Rollback: make the x264 preset the default again in the UI (instant, no deploy). For a full rollback, revert the Dockerfile and run deploy.sh again.

9. Known Limitations

- **No QSV decoding with the legacy runtime.** `--enable-qsv-decoding` fails with *"[VAAPI] No VA display found for device 0"*: on Linux, HandBrake passes the adapter index as FFmpeg's child_device, where VAAPI expects a DRM path (libhb/hwaccel.c). Upstream master behaves the same. Leave QSV decoding off in presets on this host.
- **Media SDK is end-of-life.** libmfx1 22.5.4 is Intel's final release and comes from Ubuntu 24.04. It works with the 26.04 iHD driver today, but future driver updates are not guaranteed to support it.
- **Long first build.** About 45-60 min on the i3; cached afterwards until HANDBRAKE_VERSION changes or the build cache is pruned.
- **No Dolby Vision metadata (libdovi).** It needs a Rust toolchain in the build stage; not relevant for the current sources.

10. Recommendations and Follow-ups

- Run a few real episodes with the QSV preset and compare them visually with x264. Raise the bitrate to ~1700 kbps if any difference is noticeable.
- Watch upstream HandBrake for QSV fixes on Media SDK 1.x runtimes (e.g. commit a24bfa3f1, after 1.11.2). When updating, bump HANDBRAKE_VERSION and re-test with a 2-minute segment before switching the default.
- Remove the test image `hb-qsvtest:1.11.2` (969 MB) from asus-serv-1 once the deploy is verified.
- Optional: fix the outdated description of the x264 preset ("h265, turbo fast, 720p25, 1000kbps").